

In-Class Exercise/Demo



In-Class Demo

- Let's practice `react-router-dom` using v6
V6 is the current version of react-router.
- For this Demo, we will create everything from scratch and work in `app.js` file only.
- First, create a new Github repository **m6-w3-d2-exercise** and clone this in your WESTCLIFF folder. Launch VS Code and navigate to the clone folder.
- Next create a react app: `npx create-react-app react-router-example`
- Now cd into `react-router-example`
- Next, we will add `react-router-dom` (or `react-router-dom@next` or `react-router-dom@latest`) which means - give me the most up to date version, even if it is beta):
`npm i history react-router-dom`
- Next, let's check out `package.json` to verify our dependencies.



In-Class Demo

- Now let's open `App.js` in `src` folder
- Let's import `BrowserRouter`, by adding to rest of the import
`import { BrowserRouter as Router } from 'react-router-dom';`
- Next update `App` function to read

```
function App() {  
  return <Router> {  
    } </Router>;  
}
```



In-Class Demo

- Let's import `Routes` and `Route`, update the import
`import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';`
- Next, let's add `Home` function after `App` function

```
function Home() {  
  return (  
    <div style={{ padding: 20 }}>  
      <h2>Home View</h2>  
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>  
    </div>  
  );  
}
```



In-Class Demo

- Now let's update `App` function and add `home` route

```
function App() {  
  return (  
    <Router>  
      <Routes>  
        <Route path="/" element={<Home />} />  
      </Route>  
    </Routes>  
  </Router>  
);  
}
```



In-Class Demo

- Let's run `npm start` to test what we have done so far. <http://localhost:3000/> on the browser opens.

Home View

Lorem ipsum dolor sit amet, consectetur adipiscing.



In-Class Demo

- Next, let's add [About](#) function and update [Route](#) for it

```
function About() {  
  return (  
    <div style={{ padding: 20 }}>  
      <h2>About View</h2>  
      <p>Lorem ipsum dolor sit amet, consectetur adip.</p>  
    </div>  
  );  
}
```

```
<Route path="/" element={<Home />} />  
<Route path="/about" element={<About />} />
```



In-Class Demo

- Let's check the browser again. This time go to <http://localhost:3000/about>

About View

Lorem ipsum dolor sit amet, consectetur adipiscing.



In-Class Demo

- Next, we will add [Navigation](#) to make sure we do not have to manually type in the path.
- Let's import [link](#), within react-router-dom
 - `import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';`
- Next update [App](#) function and add to router

```
<Router>
  <nav style={{ margin: 10 }}>
    <Link to="/" style={{ padding: 5 }}>
      Home
    </Link>
    <Link to="/about" style={{ padding: 5 }}>
      About
    </Link>
  </nav>
</Router>
```

Rest of the code will stay same within `<routes>` `</routes>`



In-Class Demo

- Let's check the browser again. You should see [navigation](#) that is clickable.

[Home](#) [About](#)



In-Class Demo

- Let's import **Outlet**, within react-router-dom
- In React Router v5, nested routes have to be defined explicitly. This is not the case with React Router v6. It picks one of the best elements from the React Router library called Outlet to render any matching children for a particular route.

```
import { BrowserRouter as Router, Routes, Route, Link, Outlet } from 'react-router-dom';
```



In-Class Demo

- Next, Let's create our [Blog post](#) and create a unique slug for it by [passing parameters](#)

Let's create 2 blog posts:

```
const BlogPosts = {
  '1': {
    title: 'First Blog Post',
    description: 'Lorem ipsum dolor sit amet, consectetur adipiscing elit.'
  },
  '2': {
    title: 'Second Blog Post',
    description: 'Hello React Router v6'
  }
};
```



In-Class Demo

- Next, create `Posts` and `PostLists` function

```
function Posts() {  
  return (  
    <div style={{ padding: 20 }}>  
      <h2>Blog</h2>  
      {/* render any matching child */}  
      <Outlet />  
    </div>  
  );  
}
```

```
function PostLists() {  
  return (  
    <ul>  
      {Object.entries(BlogPosts).map(([slug, { title  
    ]]) => (  
        <li key={slug}>  
          <Link to={`/${slug}`}>  
            <h3>{title}</h3>  
          </Link>  
        </li>  
      )}}  
    </ul>  
  );  
}
```

`Object.entries()` method to return an array from the object `BlogPosts`. This array is then mapped to display a list of titles of all blog posts



In-Class Demo

- Next update [App](#) function and add to route

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/posts" element={<Posts />}>
    <Route path="/posts" element={<PostLists />} />
  </Route>
</Routes>
```

Note: The PostLists route is nested within the Posts route. Rest of the code stays the same.

- Next go to browser and check the [posts](#) page



In-Class Demo

- Next we will pass parameters and create posts a link by exposing `<link>` and `{slug}`
- Let's update the `<li>` in `PostsLists` function

```
<li key={slug}>  
  <Link to={`/${posts/${slug}}`} >  
    <h3>{title}</h3>  
  </Link>  
</li>
```

Rest of the code will stay same



In-Class Demo

- Let's import `useParams`, within `react-router-dom`
- This `hook` allows you to access any `dynamic parameters` that a particular route (or slug, in this case) may have. The dynamic parameters for each slug are going to be the `title` and the `description` of each blog post. The need to access them is to display the content of each blog post when a particular slug of a blog post is triggered as the URL in the browser window

```
import { BrowserRouter as Router, Routes, Route, Link, Outlet, useParams } from  
'react-router-dom';
```




In-Class Demo

- Next we will add a function called `Post`
- Let's add this in between `Posts` and `PostLists` for consistency.

```
function Post() {  
  const { slug } = useParams();  
  const post = BlogPosts[slug];  
  const { title, description } = post;  
  return (  
    <div style={{ padding: 20 }}>  
      <h3>{title}</h3>  
      <p>{description}</p>  
    </div>  
  );  
}
```



In-Class Demo

- Finally, we can update the Route and add a link to make sure it shows in navigation.

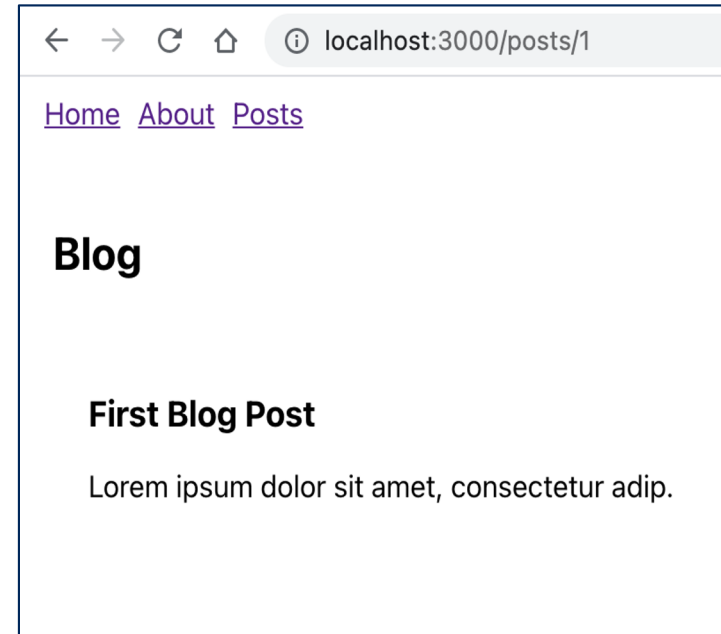
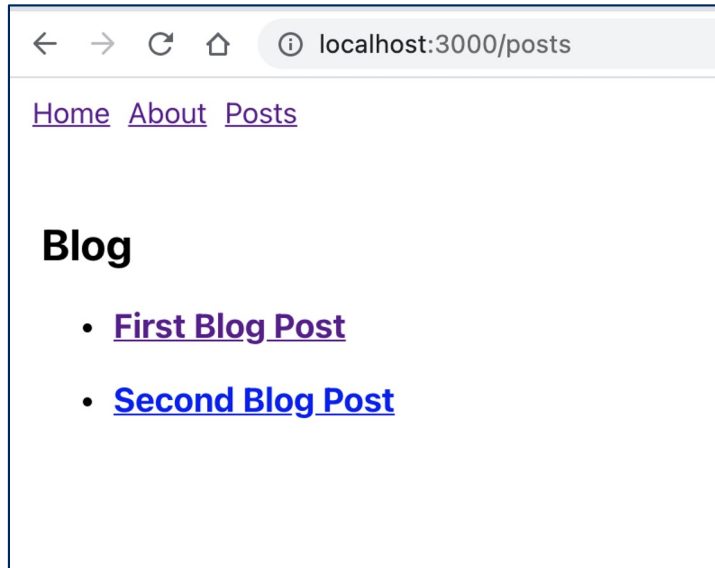
```
<Route path="/posts" element={<Posts />}>
  <Route path="/posts" element={<PostLists />} />
  <Route path=":slug" element={<Post />} />
</Route>

<Link to="/posts" style={{ padding: 5 }}>
  Posts
</Link>
```



In-Class Demo

- End result should look like this.





In-Class Exercise Demo

DUE:

Today 10.30PM PT

Submission:

Post Github link on GAP Week 7 Day 2 Exercise